Detecting darts on a dartboard with HSV colour classes

A step-by-step walkthrough of a classical computer-vision pipeline: no training data, just colour statistics, morphology and GrabCut. Input: a single photo of a Nodor Supamatch 3 board with three darts (1200 x 1600 px).

Overview

#	Step	Purpose
1	Board mask	Restrict analysis to the scoring area
2	HSV cone features	Colour space without hue wrap-around artefacts
3	Robust class models + outlier class	Black / beige / green / red; everything else = outlier
4	Edge-preserving pre-filter	Remove sisal texture speckle
5	Opening + local regrow	Remove thin wires and anti-aliased edges
6	Minimum blob area	Remove small leftovers
7	Hole fill + GrabCut	Recover the full dart region

Step 1: Board mask

Find the bullseye as the roughly circular green blob of the right size (centre at about x=624, y=873). Collect saturated red and green pixels between 300 and 470 px from the centre, keep the outermost one per 2-degree angle bin, and fit an ellipse to them with `cv2.fitEllipse`. An ellipse rather than a circle accounts for the camera not being square-on to the board. The ellipse is enlarged by 2% and used as the mask (about 597k pixels).

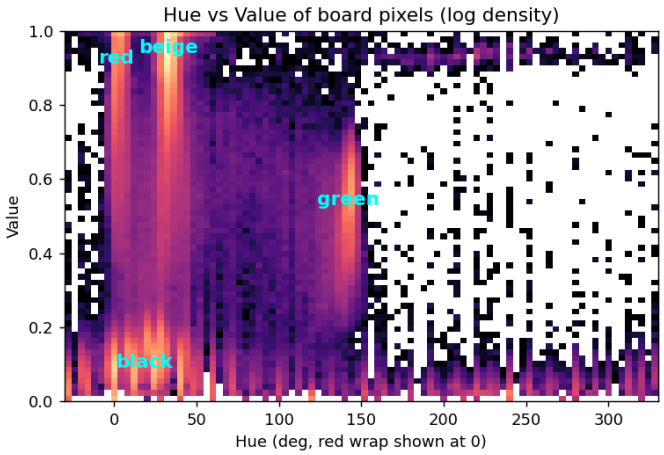


Fitted outer double ring (magenta) and detected bull centre (yellow).

Step 2: HSV cone features and the four colour peaks

Plain HSV causes two problems: hue wraps around (red sits at both 0 and 360 degrees), and hue is meaningless for near-black or near-white pixels. Each pixel is therefore mapped into the HSV cone: `x = S·V·cos(H), y = S·V·sin(H), z = V`. Dark pixels collapse towards the tip of the cone regardless of their noisy hue.

The board has four colour peaks, but no single 1D histogram shows all four: black sisal is actually a very dark warm brown, so it shares beige's hue and only separates by value; red and beige share high value and separate by saturation. In the Hue vs Value plane all four are distinct.

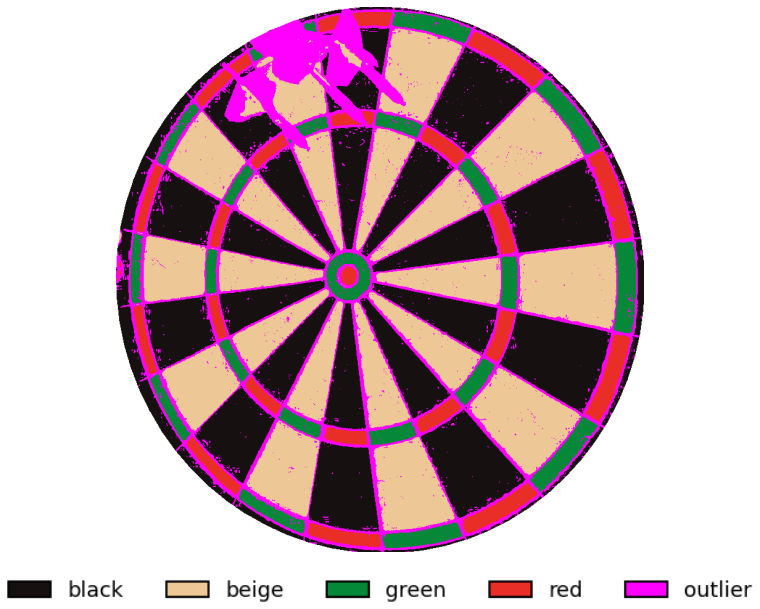


Cluster centres (HSV): black 18°/0.34/0.09 (≈40% of pixels), beige 33°/0.37/0.94 (≈37%), green 142°/0.96/0.53 (≈8%), red 1°/0.83/0.91 (≈7%).

Step 3: Robust class models and the outlier class

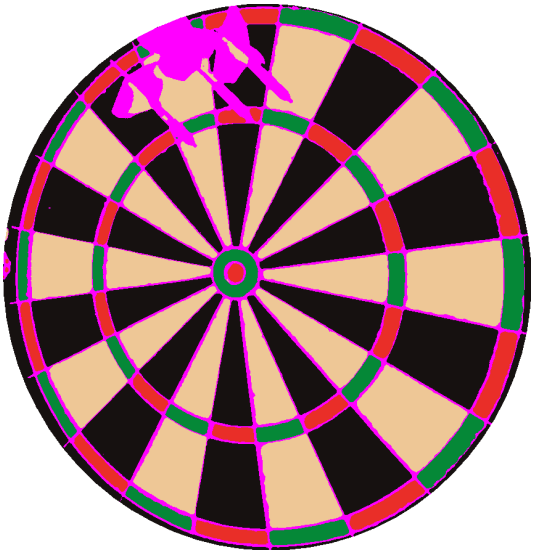
Each pixel is first assigned to the nearest of the four centres. For each class, a Gaussian is fitted to 20,000 sampled pixels with `sklearn.covariance.MinCovDet` (support fraction 0.75). The robust estimator ignores the blended edge pixels that would otherwise inflate the covariance.

A pixel keeps its class only if its squared Mahalanobis distance to the nearest class is within the chi-square (3 dof) 99.9% quantile (16.27). Otherwise it goes to a fifth **outlier** class. The darts now appear as outliers, but so do wires, segment edges and texture speckle: 16.2% of the board.



Step 4: Edge-preserving pre-filter

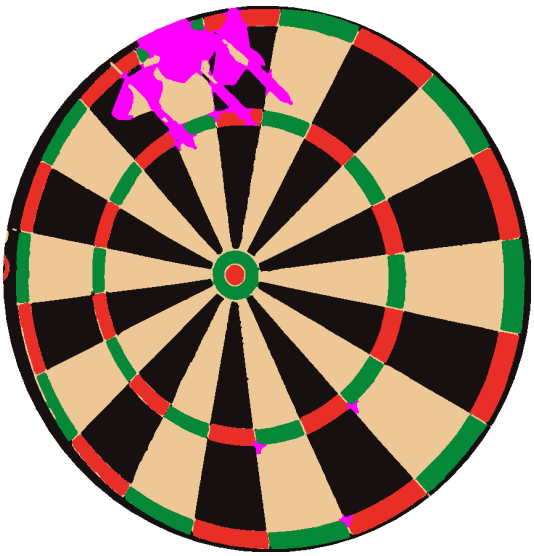
Apply a 5 px median filter followed by a bilateral filter (d=9, sigmaColor=40, sigmaSpace=7). This smooths the cracks and fibres of the sisal while keeping the boundaries between segments sharp. Important detail: the class models from Step 3 are fitted on the *raw* image and applied to the filtered one, so filtering pulls pixels towards their class centres instead of also shrinking the classes. Outliers drop to 12.9%.



Step 5: Opening and local regrow

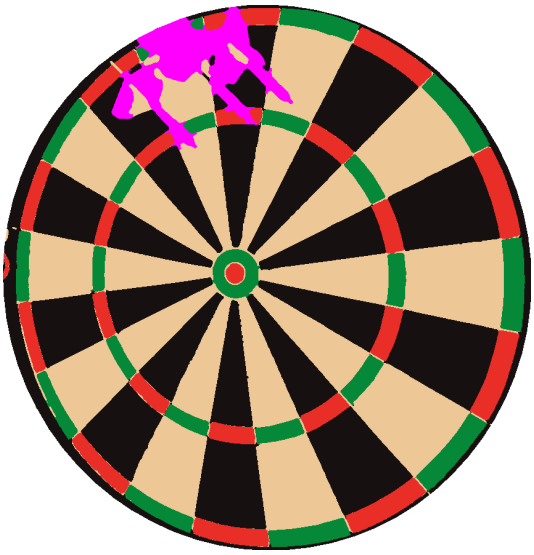
The remaining noise is thin: wires and anti-aliased edges are only a few pixels wide. A morphological opening with an 11 px elliptical kernel deletes anything thinner than the kernel, while the solid dart blobs survive. The survivors are then dilated by 15 px and intersected with the original outlier mask, so they regain their original outline.

Note: a full morphological reconstruction does *not* work here, because the edge lines connect every outlier into one network and reconstruction would restore all of it. Outliers drop to 4.0%. Pixels removed from the outlier class are returned to their nearest colour class.



Step 6: Minimum blob area

Connected components smaller than 1500 px are dropped (logo fragments, a spot near a wire), followed by a 7 px closing. The result is a single blob of about 23.3k px: the three darts touch because their flights overlap in the photo. Outliers: 3.9%, all of them darts.



Step 7: Hole fill and GrabCut reconstruction

Colour alone cannot capture the whole dart: brightly lit faces of the white flights match the beige sisal, and the flight tips extend beyond the board mask. The blob is therefore used as a seed for `cv2.grabCut` (8 iterations, mask initialisation):

- sure foreground: the hole-filled blob eroded by 9 px
- probable foreground: the hole-filled blob
- probable background: a 45 px band around it (allowed to extend past the board ellipse)
- background: everything else

GrabCut learns its own colour model for the darts and uses image edges to place the boundary. Afterwards only components touching the seed are kept, followed by a 5 px closing and hole fill. The dart region grows from 23.3k to 25.4k px and now includes the full flights, the tips and the barrels.



Before: colour-outlier blob after Step 6



After: hole fill + GrabCut

Results

Stage	Outlier pixels (% of board)
Step 3: raw, 99.9% threshold	16.2%
Step 4: + pre-filter	12.9%
Step 5: + opening and regrow	4.0%
Step 6: + minimum area	3.9% (darts only)
Step 7: GrabCut	25.4k px dart region

Limitations and next steps

• Hole filling may include sisal that is completely enclosed between overlapping darts.

• Very thin stems right at the boundary can be slightly clipped.

• GrabCut needs a seed: a dart with no colour-outlier pixels would be missed.

• Overlapping darts form one blob. A watershed on the distance transform, or locating the barrel tips, would separate individual darts.

• For repeated throws or video, differencing against a reference frame of the empty board is the most robust detector; GrabCut can then refine the edges.

Tools: Python, OpenCV, NumPy, SciPy, scikit-learn, scikit-image, Matplotlib.